TUTORIAL DE GIT

# Capítulo 3. Ramificaciones en Git

**Juan Felipe Castañeda Rodríguez** · Fundación Universitaria Agraria de Colombia (UNIAGRARIA)

Ciencia de Datos — Actividad 2: Hands on, configuración de herramientas · Octubre de 2026

Las ramas son la característica estrella de Git. Permiten separarse de la línea principal de desarrollo para trabajar en algo sin afectarla. En Git crear y cambiar de rama es casi instantáneo, lo que fomenta usarlas constantemente.

## 3.1 ¿Qué es una rama?

Cada commit en Git guarda un puntero a la instantánea del contenido, el autor, el mensaje y un puntero al commit (o commits) padre. Una **rama** es simplemente un puntero móvil, muy liviano, que apunta a uno de esos commits. La rama por defecto suele llamarse main (antes master).

Git sabe en qué rama estás gracias a un puntero especial llamado **HEAD**.

**Crear una rama:**

```
git branch pruebas
```

Esto crea el puntero, pero no cambia de rama.

**Cambiar de rama:**

```
git switch pruebas
git checkout pruebas
```

**Crear y cambiar en un solo paso:**

```
git switch -c nueva_funcion
git checkout -b nueva_funcion
```

Al hacer commits en una rama, solo esa rama avanza. Si después se vuelve a main y se hacen otros commits, la historia se **bifurca**. Para verlo:

```
git log --oneline --decorate --graph --all
```

## 3.2 Procedimientos básicos de ramificación y fusión

Un flujo típico: se está trabajando en una tarea en la rama tarea53 y llega una corrección urgente.

```
git switch main
git switch -c correccion
# ...se corrige y se confirma...
git switch main
git merge correccion
git branch -d correccion
```

**Fusión fast-forward.** Si la rama a fusionar es descendiente directa de la actual, Git solo mueve el puntero hacia adelante.

**Fusión de tres vías.** Si las historias divergieron, Git usa los dos extremos y su ancestro común para crear un **commit de fusión** con dos padres.

**Conflictos.** Si las dos ramas cambiaron la misma parte de un archivo, Git detiene la fusión y marca el archivo así:

```
<<<<<<< HEAD
contenido de la rama actual
=======
contenido de la otra rama
>>>>>>> correccion
```

Para resolver: editar el archivo, dejar el contenido correcto, borrar los marcadores y luego:

```
git add archivo.txt
git commit
```

También se puede usar git mergetool para abrir una herramienta gráfica.

## 3.3 Gestión de ramas

```
git branch
git branch -v
git branch --merged
git branch --no-merged
git branch -d rama_fusionada
git branch -D rama_no_fusionada
git branch --move nombre_viejo nombre_nuevo
```

-d solo borra ramas ya fusionadas; -D fuerza el borrado.

## 3.4 Flujos de trabajo con ramas

**Ramas de largo recorrido.** Algunas ramas conviven siempre: main con código estable, develop con trabajo que se está probando, y se fusionan cuando el código está listo.

**Ramas puntuales (topic branches).** Ramas de vida corta para una sola función o corrección. Se crean, se trabaja, se fusionan y se borran. Facilitan revisar y descartar cambios de forma aislada.

## 3.5 Ramas remotas

Las ramas remotas son referencias al estado de las ramas en un remoto, con la forma remoto/rama (por ejemplo origin/main). Se actualizan cuando hay comunicación con el servidor.

```
git fetch origin
git push origin nueva_funcion
git switch --track origin/nueva_funcion
git branch -vv
git push origin --delete nueva_funcion
```

Una **rama de seguimiento** (tracking branch) es una rama local relacionada con una remota; así git pull y git push saben a dónde ir sin argumentos. Para establecerla al empujar por primera vez:

```
git push -u origin main
```

## 3.6 Reorganizar el trabajo (rebase)

Además de merge, existe rebase: toma los commits de una rama y los vuelve a aplicar encima de otra, dejando una historia lineal.

```
git switch experimento
git rebase main
git switch main
git merge experimento
```

El resultado final de archivos es el mismo que con merge, pero la historia queda más limpia.

**Regla de oro:** no hacer rebase de commits que ya se publicaron en un repositorio compartido, porque se reescribe la historia y se rompe el trabajo de otras personas.

## 3.7 Resumen

Las ramas en Git son punteros baratos. Se crean con git branch o git switch -c, se fusionan con git merge y se pueden reorganizar con git rebase. Saber resolver conflictos y manejar ramas remotas es clave para trabajar en equipo.

## Referencia

Chacon, S., & Straub, B. (2014). *Pro Git* (2.ª ed.). Apress. https://git-scm.com/book/es/v2